Analiza experimentală comparativă a algoritmilor de sortare

Butoi Alexandru-Gabriel

1. Introducere

Acest document prezintă o interpretare experimentală a rezultatelor obținute prin testarea mai multor algoritmi de sortare pe trei tipuri de structuri de date: `vector`, `array` și listă simplu înlănțuită. Seturile de date folosite au acoperit atât cazuri mici și medii, cât și instanțe foarte mari, până la ordinul sutelor de milioane de elemente. Scopul analizei este dublu:

1) evaluarea comportamentului practic al algoritmilor în raport cu complexitatea teoretică;
2) formularea unor concluzii utile pentru optimizare, standardizare experimentală și cercetare ulterioară.

Au fost analizate următoarele familii de algoritmi:

- Bubble Sort;
- Selection Sort;
- Binary Insertion Sort;
- Merge Sort clasic;
- Merge Sort paralel;
- QuickSort;
- IntroSort.

2. Cadru metodologic

Pentru fiecare algoritm au fost colectați timpi de execuție pentru același set de teste, iar pentru majoritatea implementărilor s-a urmărit separat comportamentul pe:

- `vector`;
- `array`;
- `listă simplu înlănțuită`.

Din perspectiva cercetării experimentale, această abordare este relevantă deoarece izolează două surse distincte de variație:

a) variația datorată algoritmului propriu-zis;

b) variația datorată structurii de date și localității în memorie.

Analiza nu se limitează la afirmații teoretice de forma $O(n^2)$ sau $O(n \log n)$, ci urmărește diferențele efective de timp, de ordinul nanosecundelor, milisecundelor, secundelor și, pentru cazurile nefavorabile, chiar al orelor sau zilelor estimate.

3. Rezultate experimentale: pattern-uri majore

3.1. Algoritmi de complexitate $O(n^2)$

Rapoartele pentru Bubble Sort și Selection Sort arată o degradare rapidă a performanței odată cu creșterea lui n. Pentru Bubble Sort, la $n = 10^5$, timpii ajung deja în zona zecilor de secunde, iar estimările pentru seturi foarte mari devin impracticabile. De exemplu, pentru $n = 10^6$, Bubble Sort pe `vector` depășește 310 secunde, iar pentru $n = 10^7$ depășește 8 ore; pentru $n = 10^8$, estimarea urcă la zeci de zile. Același tipar este observat și pentru Selection Sort, care rămâne în aceeași clasă de complexitate și produce costuri extreme pentru volume mari de date.

Din punct de vedere experimental, aceste rezultate validează direct faptul că algoritmii de ordin $O(n^2)$ sunt nerecomandați pentru volume medii și mari de date. Ei pot fi acceptabili doar pentru eșantioane foarte mici sau în contexte didactice.

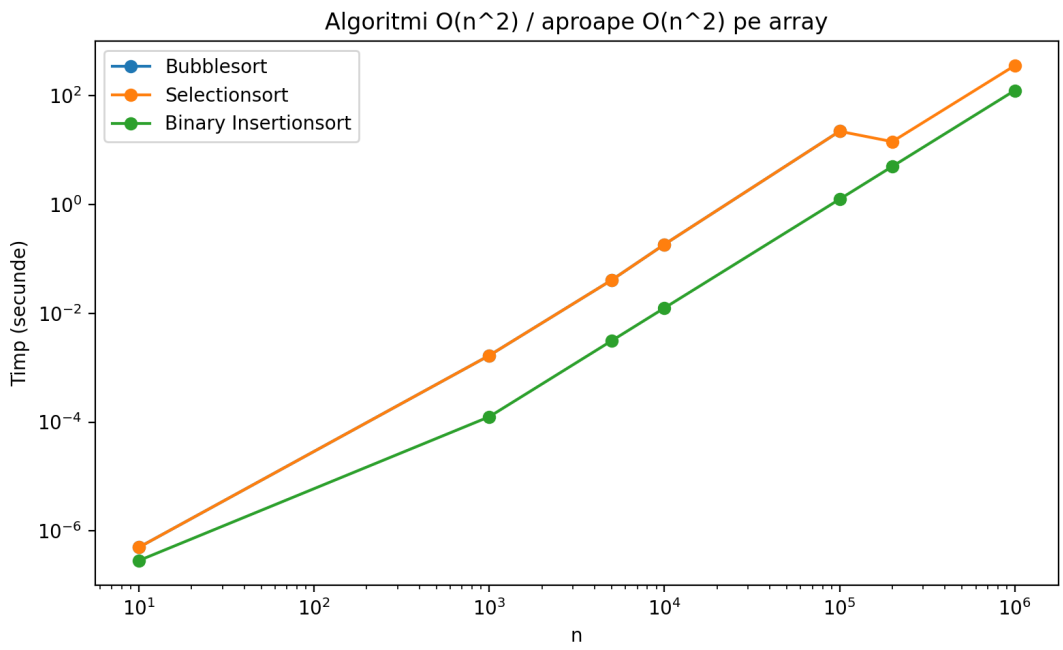

Figura 1: Evoluția timpului pentru Bubble Sort, Selection Sort și Binary Insertion Sort pe `array`, în scară logaritmică.

Graficul de mai sus arată clar că Binary Insertion Sort aduce o ameliorare față de Bubble Sort și Selection Sort pe dimensiuni mici și medii, dar nu schimbă regimul fundamental al creșterii. Tendința rămâne incompatibilă cu date de ordin foarte mare.

3.2. Algoritmi de complexitate $O(n \log n)$

Merge Sort, QuickSort și IntroSort confirmă empiric superioritatea algoritmilor de ordin $O(n \log n)$. Pentru aceiași termeni de comparație, timpii rămân în zona fracțiunilor de secundă sau a câtorva secunde chiar și atunci când n ajunge la 10^7 sau 10^8.

De exemplu, în contextul algoritmului QuickSort, pentru $n = 10^8$, timpul pe `vector` este aproximativ 4.10 secunde, iar pe `array` aproximativ 3.85 secunde. În raportul Merge Sort clasic, pentru același ordin de mărime, timpul pe `vector` este aproximativ 47.52 secunde, iar pe `array` aproximativ 17.29 secunde. În raportul IntroSort manual, pentru $n = 10^8$, timpul pe `vector` este de aproximativ 26.77 secunde, iar pe `array` de aproximativ 14.17 secunde.

Această diferență de ordin de mărime între $O(n^2)$ și $O(n \log n)$ este una dintre cele mai importante concluzii ale întregului experiment.

4. Comportamentul structurilor de date

4.1. De ce array-ul câștigă frecvent

Una dintre regularitățile cele mai clare din toate rapoartele este avantajul sistematic al `array`-ului față de `vector` și, mai ales, față de listă. Motivele sunt:

- stocarea compactă în memorie;
- localitatea spațială bună;
- accesul direct la elemente;
- penalizare minimă pentru dereferențiere.

În majoritatea algoritmilor testați, `array`-ul obține cel mai bun timp total. Diferența față de `vector` este uneori moderată, dar constantă; diferența față de listă devine adesea foarte mare.

4.2. De ce lista simplu înlănțuită pierde frecvent

Lista pierde din trei motive fundamentale:

1) accesul la elemente nu este direct, ci secvențial;
2) fiecare element este despărțit fizic în memorie de celelalte;
3) costurile de pointeri și dereferențiere se acumulează.

Prin urmare, chiar și atunci când algoritmul are teoretic aceeași complexitate asimptotică, implementarea pe listă poate fi mult mai lentă. Acest fenomen apare clar în special la Merge Sort și QuickSort, dar și la algoritmii $O(n^2)$.

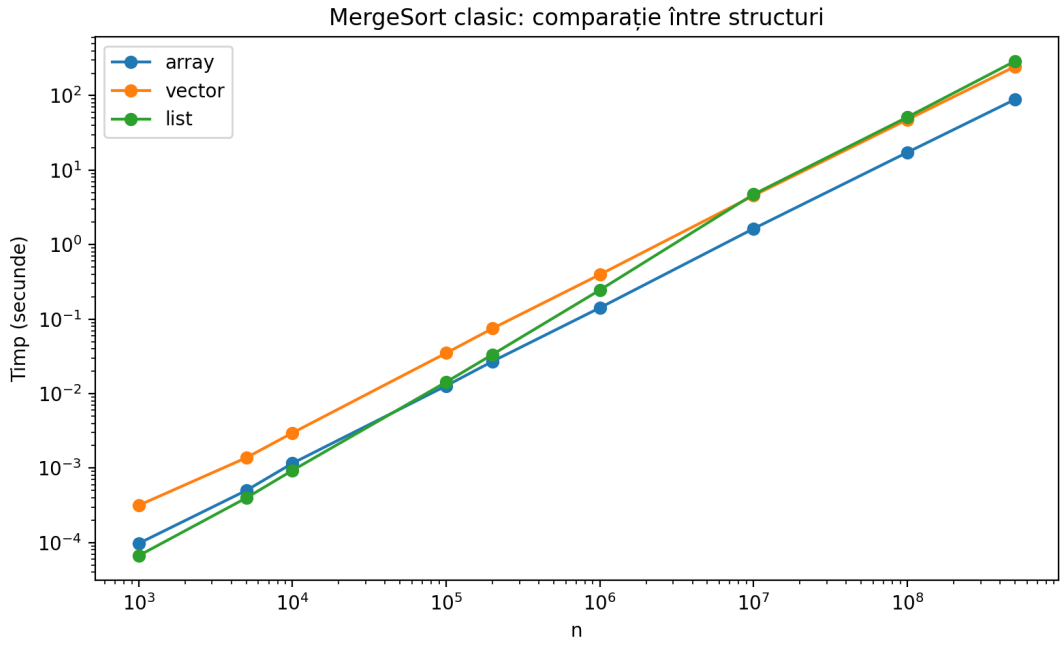

Figura 2: Merge Sort clasic: comparație între `array`, `vector` și listă simplu înlănțuită, în scară logaritmică.

Graficul evidențiază faptul că diferența dintre `array` și listă crește pe măsură ce n devine mare. Chiar dacă pe valori foarte mici lista poate părea competitivă, costurile reale de memorie și traversare o penalizează sever la scară mare.

5. Interpretări punctuale cerute de studiu

5.1. De ce MergeSort pe listă pierde timp față de MergeSort pe array

În teorie, Merge Sort funcționează bine și pe liste, deoarece interclasarea poate fi realizată elegant prin relink-area nodurilor. Totuși, rapoartele arată că implementarea pe listă este mai lentă decât cea pe `array`. Explicația principală nu ține de complexitatea asimptotică, ci de costul arhitectural al structurii.

Array-ul este cache-friendly: elementele consecutive sunt apropiate în memorie, iar procesorul preia eficient blocuri continue de date. În schimb, lista simplu înlănțuită presupune acces pointer-cu-pointer, cu dereferențieri repetate și slabă localitate de memorie. În plus, operațiile auxiliare precum împărțirea listei și traversările repetate introduc overhead suplimentar.

Experimental, diferența este clară. Pentru $n = 10^7$, Merge Sort pe `array` are aproximativ 1.64 secunde, iar pe listă aproximativ 4.73 secunde. Pentru $n = 10^8$, `array`-ul rămâne la aproximativ 17.29 secunde, în timp ce lista depășește 51.40 secunde. Pentru $n = 5 \cdot 10^8$,

`array`-ul este în jur de 88.51 secunde, iar lista urcă la aproximativ 289.41 secunde. Rezultatul este că, deși Merge Sort este natural pentru liste din punct de vedere conceptual, în practică array-ul rămâne net superior pentru volume mari de date.

5.2. De ce MergeSort paralel pe 16 thread-uri pierde față de QuickSort cu pivot random

Rezultatele experimentale arată că Merge Sort paralel pe 16 thread-uri nu depășește QuickSort-ul cu pivot random pentru volumele mari testate. De exemplu, pentru $n = 10^8$, Merge Sort paralel pe `vector` ajunge la aproximativ 14.01 secunde, în timp ce QuickSort pe `vector` rămâne în jur de 4.10 secunde. Pentru $n = 5 \cdot 10^8$, Merge Sort paralel pe `vector` este aproximativ 70.04 secunde, iar QuickSort aproximativ 23.50 secunde.

Din perspectivă de cercetare a performanței, această situație se explică prin următoarele:

- paralelizarea introduce costuri de creare și sincronizare a thread-urilor;
- interclasarea implică trafic suplimentar în memorie și copieri;
- QuickSort este in-place și necesită memorie auxiliară mult mai redusă;
- alegerea random a pivotului reduce probabilitatea de cazuri patologice și produce o partiționare eficientă în practică.

Așadar, deși paralelizarea reduce o parte din costul recursiei, ea nu elimină costul structural al algoritmului și nici overhead-ul execuției concurente. În practică, pentru implementarea testată, QuickSort rămâne mai eficient.

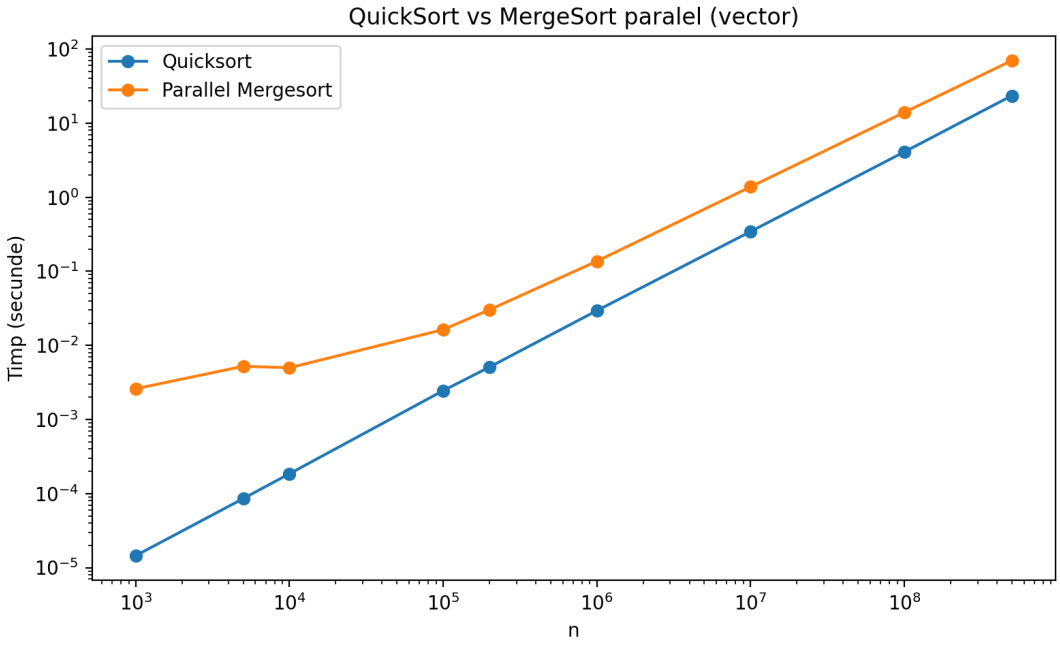

Figura 3: Comparație între QuickSort și MergeSort paralel pe `vector`, în scară logaritmică.

5.3. Păstrarea datelor pentru validare și cercetare ulterioară

Păstrarea rapoartelor brute reprezintă un pas metodologic esențial. Datele experimentale obținute nu trebuie privite doar ca ieșiri temporare ale unui program, ci ca material de cercetare reutilizabil.

Aceste date permit:

- replicarea experimentelor;
- verificarea consistenței între rulări;
- construirea de grafice de scalare;
- compararea versiunilor optimizate vs. neoptimizate;
- investigarea impactului distribuției inputului;
- calibrarea pragurilor de skip sau a limitelor experimentale.

În mod ideal, setul de rapoarte trebuie păstrat ca bază empirică pentru:

a) cercetare ulterioară asupra optimizării implementărilor;

b) comparații între paradigme de memorie;

c) analiză statistică a stabilității performanței.

5.4. Normalizarea tipurilor de date: conversia la întregi

O direcție foarte interesantă de cercetare este transformarea diferitelor tipuri de date într-o reprezentare unificată de tip întreg.

Cazul valorilor reale

Dacă pentru datele de tip `float` sau `double` este cunoscută precizia maximă de p zecimale, atunci fiecare valoare poate fi scalată prin înmulțire cu 10^p, iar apoi rotunjită și stocată ca întreg. În acest mod, sortarea se poate realiza doar pe valori întregi, evitând costurile și ambiguitățile asociate comparațiilor în virgulă mobilă.

Avantajele unei astfel de transformări sunt:

- uniformizarea codului de sortare;
- posibilitatea reutilizării aceloraşi structuri și algoritmi pe un domeniu discret;
- control mai bun asupra ordonării și asupra reprezentării.

Limitarea majoră este riscul de overflow. Dacă valoarea scalată depășește intervalul tipului folosit, transformarea devine nesigură. În practică:

- `int` pe 32 de biți este adesea insuficient pentru intervale mari sau precizie ridicată;
- `long long` pe 64 de biți extinde mult domeniul, dar nu elimină complet riscul;
- pentru valori foarte mari sau precizie mare, `double` sau un tip arbitrar de precizie rămân necesare.

Cazul caracterelor

Pentru `char`, conversia la `int` este aproape naturală, deoarece caracterul are deja o reprezentare numerică internă. Din acest motiv, sortarea caracterelor poate fi interpretată ca o sortare de coduri numerice. Totuși, din punct de vedere al eficienței, câștigul real este limitat, deoarece comparațiile pe `char` sunt deja foarte ieftine. Beneficiul principal nu ar fi viteza brută, ci simplificarea și unificarea infrastructurii de testare.

Concluzia este că transformarea tipurilor în întregi poate fi utilă ca strategie de standardizare experimentală, dar trebuie controlată riguros pentru a evita overflow-ul și pierderea de informație.

5.5. De ce implementarea manuală de IntroSort este inferioară lui `std::sort`

Deși `std::sort` este bazat pe un algoritm de tip introsort, rezultatele arată că implementarea manuală este mult mai lentă. Pentru $n = 10^8$, implementarea manuală pe `vector` are aproximativ 26.77 secunde, iar pe `array` aproximativ 14.17 secunde. În schimb, implementarea foarte optimizată din bibliotecile standard produce timpi de ordinul 4.10 secunde pe `vector` și 3.85 secunde pe `array` pentru aceeași scară experimentală.

Această diferență se explică prin faptul că `std::sort` nu este doar o idee algoritmică, ci o implementare industrială, rafinată la nivel de bibliotecă standard. Ea beneficiază de:

- praguri calibrate experimental pentru trecerea la insertion sort;
- partiționare foarte optimizată;
- management eficient al recursiei;
- compatibilitate excelentă cu optimizările compilatorului;
- reducerea overhead-ului pe ramuri și apeluri.

Prin contrast, implementarea manuală folosită în experiment este corectă conceptual, dar nu poate concura cu nivelul de rafinare al STL. Astfel, concluzia corectă nu este că introsortul este slab, ci că implementarea manuală este mai puțin favorabilă decât varianta standard extrem de optimizată.

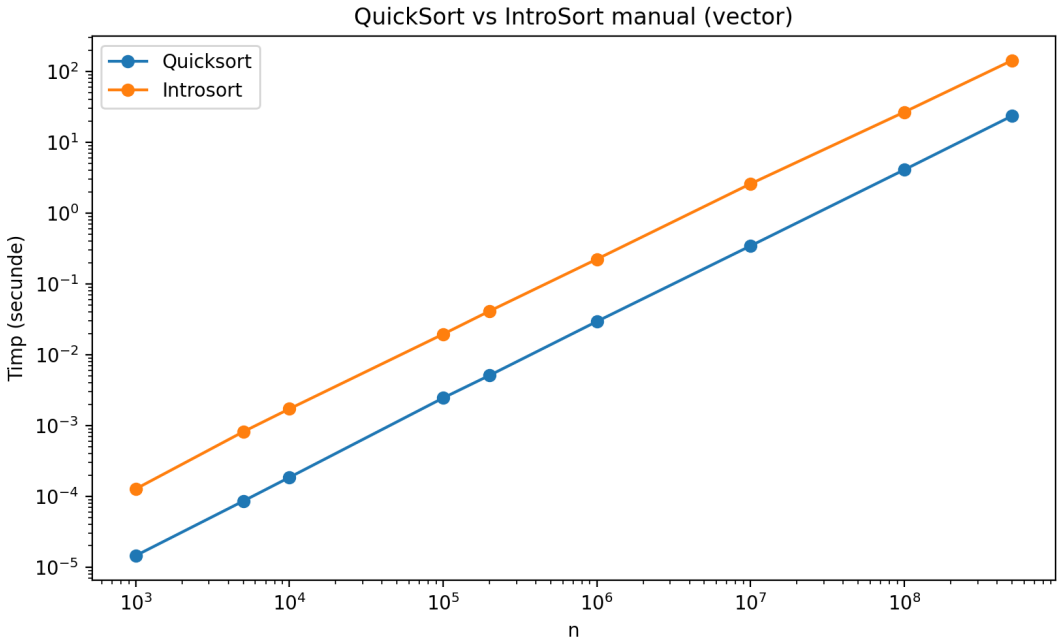

Figura 4: QuickSort vs IntroSort manual pe `vector`.

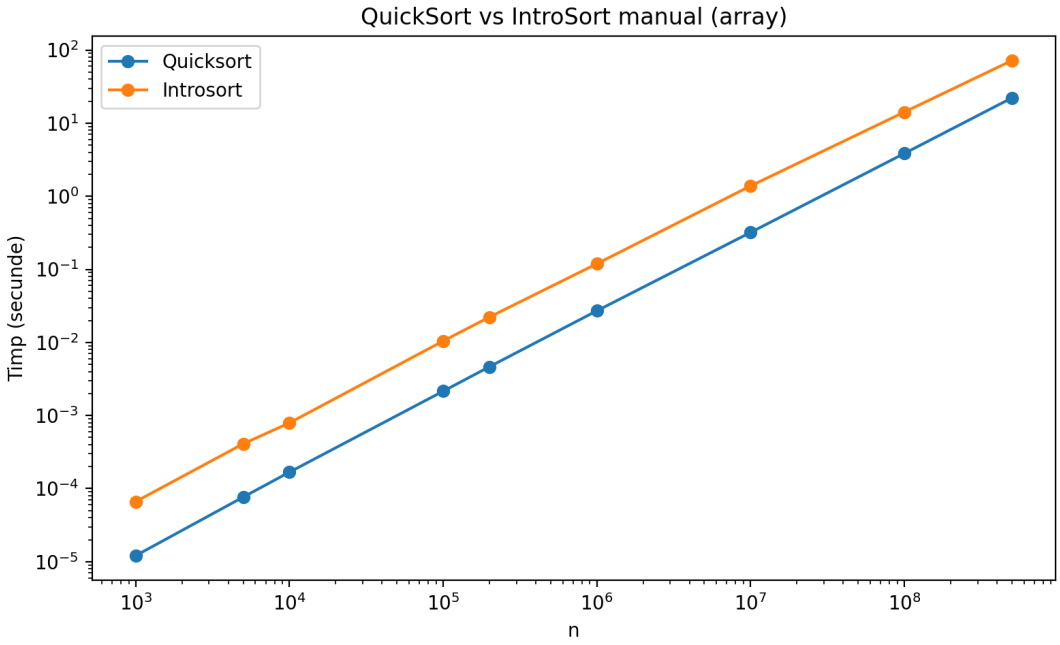

Figura 5: QuickSort vs IntroSort manual pe `array`.

6. Observații comparative asupra algoritmilor

6.1. Bubble Sort și Selection Sort

Ambele confirmă experimental costul quadratic. Sunt utile pentru studiu didactic și pentru volume foarte mici de date, dar devin rapid impracticabile. Array-ul este aproape întotdeauna cea mai bună structură, iar lista și vectorul rămân în urmă.

6.2. Binary Insertion Sort

Această metodă reduce numărul de comparații prin căutare binară, dar deplasarea elementelor păstrează costul total în $O(n^2)$. Ea poate oferi îmbunătățiri pe seturi mici și medii, dar nu schimbă regimul asimptotic.

6.3. Merge Sort clasic

Este robust, stabil și predictibil, dar costul de memorie auxiliară și copierea datelor afectează timpul final. Pe array funcționează clar mai bine decât pe listă. Pe vector rămâne bun, dar adesea inferior array-ului.

6.4. Merge Sort paralel

Este interesant ca experiment de paralelizare, însă overhead-ul său îl face mai puțin competitiv decât QuickSort în implementarea testată. Avantajele pot deveni vizibile doar în condiții bine controlate, pe hardware și inputuri foarte favorabile.

6.5. QuickSort

În implementarea cu pivot random, QuickSort oferă una dintre cele mai bune performanțe practice. Costul redus de memorie și partiționarea in-place îl fac foarte competitiv.

6.6. IntroSort

Din punct de vedere teoretic și practic, introsortul este una dintre cele mai robuste soluții. Totuși, diferența dintre implementarea manuală și cea standard demonstrează că performanța unui algoritm depinde și de calitatea concretă a implementării, nu doar de alegerea ideii algoritmice.

6.7. Sensibilitatea la distribuția datelor

Seturile generate experimental au inclus nu doar distribuții random, ci și cazuri egale, aproape sortate, reverse, cu multe duplicate sau cu interval numeric restrâns. Această diversitate arată că aceeași dimensiune n poate produce timpi diferiți în funcție de forma inputului.

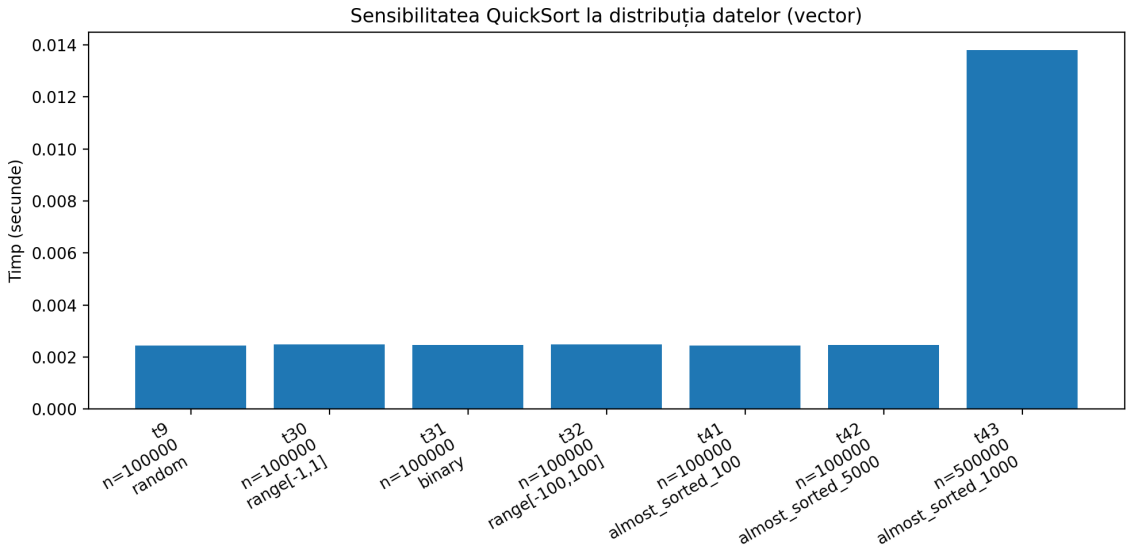

Figura 6: Sensibilitatea QuickSort la distribuția datelor pentru câteva teste reprezentative.

7. Direcții viitoare de cercetare

Rezultatele experimentale obținute în această lucrare nu epuizează analiza posibilă, ci deschid mai multe direcții de cercetare viitoare, relevante atât din perspectivă teoretică, cât și practică.

7.1. Analiza sensibilității algoritmilor la distribuția datelor

O primă direcție importantă de cercetare este studiul sistematic al sensibilității algoritmilor de sortare la distribuția datelor de intrare. Seturile de test utilizate în această lucrare au fost generate astfel încât să acopere mai multe tipuri relevante de distribuții:

- date random;
- date deja sortate;
- date sortate invers;
- date egale;
- date aproape sortate;
- date cu multe duplicate;
- date binare sau cu interval numeric foarte restrâns.

Această varietate experimentală este esențială, deoarece algoritmii nu reacționează uniform la toate distribuțiile. De exemplu, testele de tip `genEqual`, `genAlmostSorted`, respectiv cele cu interval restrâns, permit observarea unor cazuri în care anumite metode se apropie de comportamente mult mai bune decât limita lor nefavorabilă teoretică, în timp ce altele rămân sensibile la costurile structurale ale implementării. Rapoartele confirmă clar că forma distribuției influențează masiv timpul efectiv de execuție, nu doar dimensiunea lui n.

În cercetări viitoare, această direcție poate fi extinsă prin gruparea formală a testelor pe clase de distribuții și prin construirea unor profile de sensibilitate pentru fiecare algoritm.

7.2. Studiul efectului localității în memorie

Rezultatele obținute indică în mod repetat că structura de date are un impact major asupra performanței. În majoritatea algoritmilor testați, `array`-ul este mai rapid decât `vector`-ul, iar lista simplu înlănțuită este frecvent cea mai lentă structură, mai ales pentru

volume mari de date. Acest comportament sugerează o influență puternică a localității în memorie, a utilizării cache-ului și a costurilor de dereferențiere a pointerilor.

O direcție viitoare de cercetare este, prin urmare, corelarea explicită dintre timpii de execuție și caracteristicile de acces la memorie, în special pentru:

- structuri contigue (`array`, `vector`);
- structuri bazate pe pointeri (listă simplu înlănțuită);
- algoritmi care fac copieri auxiliare frecvente;
- algoritmi in-place versus algoritmi cu memorie auxiliară.

7.3. Praguri optime și algoritmi hibrizi

O altă direcție relevantă este determinarea experimentală a pragurilor optime pentru algoritmii hibrizi. În cazul IntroSort, performanța depinde nu doar de ideea de bază, ci și de momentul în care algoritmul trece de la QuickSort la HeapSort, respectiv de dimensiunea subproblemelor pentru care este introdus un algoritm auxiliar, precum Insertion Sort.

Deși implementarea manuală utilizată în această lucrare este corectă conceptual, performanțele sale rămân inferioare față de implementările foarte optimizate de tip `std::sort`, ceea ce sugerează că alegerea pragurilor și rafinarea detaliilor de implementare pot influența decisiv rezultatul final.

O continuare firească a cercetării ar fi calibrarea acestor praguri în funcție de:

- dimensiunea subsecvențelor;
- tipul de date;
- structura de stocare;
- distribuția datelor.

7.4. Strategii alternative de alegere a pivotului în QuickSort

În implementările QuickSort utilizate, alegerea pivotului a fost făcută random, ceea ce a oferit rezultate foarte bune în practică. Totuși, strategia de alegere a pivotului rămâne un factor critic în stabilitatea și robustețea algoritmului.

Prin urmare, o direcție de cercetare viitoare este compararea experimentală între:

- pivot random;

- primul element;
- ultimul element;
- median-of-three;
- alte strategii euristice de selecție.

O asemenea comparație ar permite determinarea relației dintre alegerea pivotului, distribuția datelor și stabilitatea timpului de execuție.

7.5. Raportul dintre paralelizare și beneficiul real de performanță

O direcție de cercetare deosebit de importantă este investigarea raportului real dintre paralelizare și câștigul de performanță. Teoretic, pentru volume mari de date, o implementare paralelizată a lui MergeSort ar trebui să devină foarte competitivă și, în anumite condiții, chiar să depășească QuickSort. Totuși, în experimentele prezente, acest lucru nu s-a confirmat, iar QuickSort a rămas superior în practică pe multe dintre testele mari.

Acest rezultat face tema deosebit de interesantă pentru cercetare, deoarece sugerează existența unui decalaj între potențialul teoretic al paralelizării și comportamentul practic al implementării testate. Tocmai această discrepanță justifică aprofundarea ulterioară a subiectului, prin analiza unor factori precum:

- costul real al creării și sincronizării thread-urilor;
- pragul de la care paralelizarea devine rentabilă;
- distribuția sarcinilor între thread-uri;
- costul memoriei auxiliare și al interclasării.

Această direcție nu trebuie tratată ca o invalidare a ideii de MergeSort paralel, ci ca un semnal clar că implementarea practică și mediul de execuție influențează decisiv rezultatul final.

7.6. Separarea riguroasă a timpului de sortare de costul I/O

Din punct de vedere metodologic, o observație importantă în favoarea validității experimentului este faptul că măsurarea timpului a fost realizată corect: citirea datelor este efectuată înainte de pornirea cronometrării, timerul este pornit exact înainte de apelul funcției de sortare, iar oprirea sa are loc imediat după finalizarea sortării, înainte de scrierea rezultatului în fișier. Prin urmare, valorile măsurate reflectă timpul algoritmului de sortare propriu-zis, nu timpul combinat de I/O și procesare.

Această alegere experimentală este foarte importantă și reprezintă deja un punct solid al cercetării, deoarece izolează variabila relevantă. În cercetări viitoare, această separare ar putea fi dusă și mai departe prin raportarea distinctă a:

- timpului de citire;
- timpului efectiv de sortare;
- timpului de scriere.

Totuși, pentru obiectivul actual al lucrării, metoda folosită este corectă și suficient de riguroasă.

7.7. Normalizarea reprezentării datelor

O altă direcție posibilă, formulată ca ipoteză de lucru, este normalizarea reprezentării tipurilor de date. În cazul valorilor reale, dacă se cunoaște numărul de zecimale generate, se poate aplica o scalare de forma $x \mapsto x \cdot 10^p$, după care valorile pot fi tratate ca numere întregi. În mod similar, pentru `char`, conversia la cod numeric întreg este naturală.

Această idee ar putea simplifica uniformizarea infrastructurii experimentale și ar permite reutilizarea aceloraşi implementări pe un domeniu comun discret. Totuși, analiza detaliată a acestei direcții depășește scopul lucrării de față și poate constitui o temă separată de investigație, mai ales în raport cu limitele de reprezentare și riscul de overflow.

7.8. Observații privind mediul de execuție

Toate testele au fost realizate în același mediu de execuție, folosind WSL în VSCode. Din punct de vedere experimental, acest aspect reprezintă un avantaj metodologic important: toate algoritmii au fost evaluați în condiții uniforme, ceea ce face comparațiile interne valide și relevante.

Prin urmare, chiar dacă valorile absolute ale timpilor pot depinde de mediul de execuție, comparația relativă dintre algoritmi, structuri și distribuții de date rămâne semnificativă. Pentru această lucrare, consistența mediului este mai importantă decât diversitatea platformelor testate, deoarece obiectivul principal este comparația experimentală controlată.

8. Concluzie privind direcțiile viitoare

În ansamblu, direcțiile viitoare de cercetare trebuie să urmărească nu doar optimizarea unor implementări individuale, ci și înțelegerea relației dintre:

- complexitatea teoretică;
- distribuția datelor;
- structura de date;
- localitatea în memorie;
- costurile auxiliare ale implementării;
- mediul concret de execuție.

Această abordare permite transformarea unui studiu comparativ de algoritmi într-o cercetare experimentală mai profundă, orientată către comportamentul real al metodelor de sortare în practică.

9. Concluzii finale

Pe baza întregului set de rapoarte, se pot formula următoarele concluzii de cercetare:

1) Algoritmii $O(n^2)$ sunt improprii pentru volume mari de date, indiferent de structură.
2) Algoritmii $O(n \log n)$ reprezintă alegerea corectă pentru seturi mari.
3) Array-ul este, în mod constant, cea mai eficientă structură pentru sortare.
4) Lista simplu înlănțuită suferă din cauza localității slabe în memorie.
5) Paralelizarea nu garantează superioritate practică; overhead-ul poate anula beneficiul.
6) Standardizarea datelor și păstrarea rapoartelor sunt esențiale pentru cercetare ulterioară.
7) Implementările STL trebuie tratate ca etalon practic, nu doar ca referință teoretică.

În concluzie, analiza experimentală confirmă că alegerea algoritmului optim nu depinde doar de formula teoretică a complexității, ci de interacțiunea dintre:

- algoritm;
- structură de date;
- reprezentarea tipului de date;

- implementare;
- arhitectura de memorie;
- costul practic al operațiilor auxiliare.